

cpp\Homework.cpp

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4
5  struct nodeType{
6      int info;
7      nodeType *next;
8  };
9
10 nodeType* initializeList(){
11     return nullptr;
12 }
13
14 nodeType* getNode(){
15     nodeType *p = new nodeType;
16     return p;
17 }
18
19 void freeNode(nodeType *p){
20     delete p;
21 }
22
23 nodeType* createList(nodeType *plist, int n){
24     nodeType *p, *ptr;
25     int node;
26     cout<<"Enter node 0: "; cin>>node;
27     p = getNode(); p->info = node; p->next = nullptr;
28     plist = p; ptr = plist;
29     for(int i=1;i<n;i++){
30         cout<<"Enter node "<<i<<": "; cin>>node;
31         p = getNode(); p->info = node; p->next = nullptr;
32         ptr->next = p; ptr = p;
33     }
34     return plist;
35 }
36
37 void traverse(nodeType *plist){
38     nodeType *p = plist;
39     cout<<"List: ";
40     while(p!=nullptr){ cout<<p->info<<" -> "; p=p->next; }
41     cout<<"NULL\n";
42 }
43
44 int countNode(nodeType *plist){
45     int count = 0;
46     for(nodeType *p=plist;p!=nullptr;p=p->next) count++;
47     return count;
48 }
```

```

49
50 nodeType* searchPos(nodeType *plist, int node){
51     for(nodeType *p=plist;p!=nullptr;p=p->next)
52         if(p->info==node) return p;
53     return nullptr;
54 }
55
56 void sequentialSearch(nodeType *plist, int node){
57     int pos = 0;
58     for(nodeType *p=plist;p!=nullptr;p=p->next){
59         if(p->info==node){ cout<<"Found '"<<node<<"' at position "<<pos<<".\n"; return; }
60         pos++;
61     }
62     cout<<"Not found!\n";
63 }
64
65 void binarySearch(nodeType *plist, int node){
66     int n = countNode(plist);
67     int arr[100];
68     int i = 0;
69     for(nodeType *p=plist;p!=nullptr;p=p->next) arr[i++]=p->info;
70     for(int a=0;a<n-1;a++)
71         for(int b=0;b<n-a-1;b++)
72             if(arr[b]>arr[b+1]){ int tmp=arr[b]; arr[b]=arr[b+1]; arr[b+1]=tmp; }
73     int left=0, right=n-1;
74     while(left<=right){
75         int mid=(left+right)/2;
76         if(arr[mid]==node){ cout<<"Found '"<<node<<"' at position "<<mid<<" (sorted
array).\n"; return; }
77         else if(arr[mid]<node) left=mid+1;
78         else right=mid-1;
79     }
80     cout<<"Not found!\n";
81 }
82
83 void sortNode(nodeType *plist){
84     int tmp;
85     for(nodeType *p=plist;p!=nullptr;p=p->next)
86         for(nodeType *ptr=p->next;ptr!=nullptr;ptr=ptr->next)
87             if(p->info>ptr->info){ tmp=p->info; p->info=ptr->info; ptr->info=tmp; }
88 }
89
90 nodeType* insertNode(nodeType *plist, int node){
91     nodeType *p = getNode(); p->info = node; p->next = nullptr;
92     if(plist==nullptr){ plist=p; }
93     else{ nodeType *ptr=plist; while(ptr->next!=nullptr) ptr=ptr->next; ptr->next=p; }
94     return plist;
95 }
96
97 bool updateNode(nodeType *plist, int oldNode, int newNode){

```

```
98     nodeType *target = searchPos(plist, oldNode);
99     if(target==nullptr) return false;
100    target->info = newNode;
101    return true;
102 }
103
104 nodeType* deleteNode(nodeType *plist, int node){
105     nodeType *p=plist, *prev=nullptr;
106     while(p!=nullptr && p->info!=node){ prev=p; p=p->next; }
107     if(p==nullptr){ cout<<"Node not found!\n"; return plist; }
108     if(prev==nullptr) plist=p->next;
109     else prev->next=p->next;
110     freeNode(p);
111     return plist;
112 }
113
114 void saveToFile(nodeType *plist){
115     ofstream f("list1.txt");
116     for(nodeType *p=plist;p!=nullptr;p=p->next) f<<p->info<<" ";
117     f.close();
118 }
119
120 nodeType* loadFromFile(nodeType *plist){
121     ifstream f("list1.txt");
122     int node;
123     while(f>>node) plist=insertNode(plist,node);
124     f.close();
125     return plist;
126 }
127
128 int main(){
129     nodeType *plist = initializeList();
130     int n, option;
131     cout<<"GROUP 12\n";
132     cout<<" - Bros Sophary\n - Phan Vichararoth\n - Maeng Chansreyha\n";
133     do{
134         cout<<"==== MENU ==== \n";
135         cout<<"1. Create\n2. Insert\n3. Display\n4. Search\n5. Sort\n6. Delete\n7. Update\n8.
Count\n9. Save\n10. Load\n0. Exit\n";
136         cout<<"Choose Option: "; cin>>option;
137         switch(option){
138             case 1:{
139                 cout<<"Enter number of nodes: "; cin>>n;
140                 plist=createList(plist,n);
141                 cout<<"List created.\n";
142                 break;
143             }
144             case 2:{
145                 int node;
146                 cout<<"Enter node: "; cin>>node;
```

```
147         plist=insertNode(plist,node);
148         cout<<"Inserted successfully.\n";
149         break;
150     }
151     case 3:{
152         traverse(plist);
153         break;
154     }
155     case 4:{
156         int node;
157         int s;
158         cout<<"--- Search Menu ---\n";
159         cout<<"1. Sequential Search\n2. Binary Search\n";
160         cout<<"Choose: "; cin>>s;
161         cout<<"Enter node: "; cin>>node;
162         if(s==1) sequentialSearch(plist,node);
163         else if(s==2) binarySearch(plist,node);
164         else cout<<"Invalid choice!\n";
165         break;
166     }
167     case 5:{
168         sortNode(plist);
169         cout<<"Sorting complete (Bubble Sort).\n";
170         break;
171     }
172     case 6:{
173         int node;
174         cout<<"Enter node to delete: "; cin>>node;
175         plist=deleteNode(plist,node);
176         cout<<"Deleted successfully.\n";
177         break;
178     }
179     case 7:{
180         int oldNode, newNode;
181         cout<<"Enter node to update: "; cin>>oldNode;
182         cout<<"Enter new node: "; cin>>newNode;
183         if(updateNode(plist,oldNode,newNode)) cout<<"Updated successfully.\n";
184         else cout<<"Node not found!\n";
185         break;
186     }
187     case 8:{
188         cout<<"Total nodes: "<<countNode(plist)<<"\n";
189         break;
190     }
191     case 9:{
192         saveToFile(plist);
193         cout<<"Data saved to file.\n";
194         break;
195     }
196     case 10:{
```

```
197         plist=loadFromFile(initializeList());
198         cout<<"Data loaded from file.\n";
199         traverse(plist);
200         break;
201     }
202     case 0:{ break; }
203     default:{ cout<<"Invalid option!\n"; break; }
204 }
205 }while(option!=0);
206 return 0;
207 }
```